

WiDS Kalman Filtered Trend Trader

Mid-Term Report

Udit Mittal

Project I'd - 36

Foundations of Machine Learning

Built a rigorous conceptual and mathematical understanding of core machine learning methods such as linear and nonlinear regression models, neural network architectures, and deep learning techniques.

Simple Linear Regression -

In this part, we derived the values of the coefficients of simple linear regression (SLR) lines i.e. β_0 and β_1 . Where they are co-related by this equation.

$$\hat{y} = \beta_0 + \beta_1 \cdot x$$

Here x and y are variables between which we are trying to develop a linear relation. Now we want to create such a model such that our error is minimum. Therefore, we find Sum of Squared Errors, and not sum of modulus of errors because squaring gives amplification to all the large errors, on which we want to focus and also the function is continuous and differentiable everywhere unlike modulus function. So, in order to find minimum error-

$$e_i = y_i - \hat{y}_i$$

$$e_i = y_i - (\beta_0 + \beta_1 \cdot x_i)$$

$$SSE = \sum e_i^2$$

Now,

$$\frac{\partial(SSE)}{\partial\beta_0} = 0 \text{ and } \frac{\partial(SSE)}{\partial\beta_1} = 0 .$$

Now on solving these both partial derivatives gives the value of β_0 and β_1 .

$$\begin{aligned} \sum y_i &= n\beta_0 + \beta_1 \sum x_i \\ \sum x_i y_i &= \beta_0 \sum x_i + \beta_1 \sum x_i^2 \\ \beta_1 &= \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2}, \quad \beta_0 = \bar{y} - \beta_1 \bar{x} \end{aligned}$$

Substituting these values back into the regression equation gives,

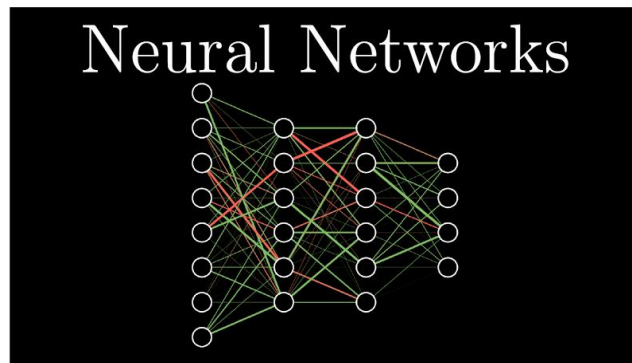
$$\hat{y} = \beta_0 + \beta_1 \cdot x$$

the **best-fit line** for the given dataset. This line minimizes the sum of squared errors, ensuring that the overall deviation between the observed values y_i and the predicted values $\hat{y}_i$ is as small as possible. These two parameters completely define the simple linear regression model and can be computed directly from the dataset using the above formulas.

Neural Networks -

A neural network is a function approximator composed of layers of interconnected **neurons** (or nodes). Basic architecture is composed of –

- Input Layer
- Hidden Layer(s)
- Output Layer



Weighted sum at neuron:

$$z = \sum_{i=1}^n w_i x_i + b$$

where x_i = input, w_i = weight, and b = bias.

Activation Functions:

Activation functions introduce **non-linearity** into a neural network, allowing it to learn complex patterns. Without activation functions, a neural network would behave like a linear model regardless of the number of layers.

Given a neuron input

$$z = \sum_i w_i x_i + b$$

the activation function transforms z into the neuron output $a = f(z)$.

Common Activation Functions

- **Sigmoid**

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- Output range: (0, 1)
- Used for probability-like outputs
- Limitation: vanishing gradients for large $|z|$

- **ReLU (Rectified Linear Unit)**

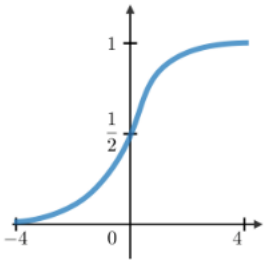
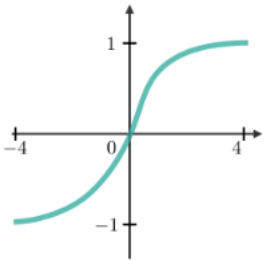
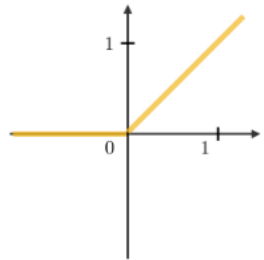
$$\text{ReLU}(z) = \max(0, z)$$

- Most commonly used in hidden layers
- Efficient and reduces vanishing gradient problem
- Limitation: “dead neurons” for negative inputs

- **Tanh**

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

- Output range: (-1, 1)
- Zero-centered, better than sigmoid in many cases

Sigmoid	Tanh	ReLU
$g(z) = \frac{1}{1 + e^{-z}}$	$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$g(z) = \max(0, z)$
		

Activation functions enable neural networks to approximate **non-linear functions**, which is essential for tasks like image recognition, speech processing, and pattern classification.

Forward Propagation

The process of computing the output given an input:

- Multiply inputs by weights
- Add biases

- Apply activation function

$$a^{(l)} = f(W^{(l)}a^{(l-1)} + b^{(l)})$$

where $a^{(l)}$ = activations of layer l .

Learning & Optimization:

Loss Function

Measure of how far the predictions are from actual labels.

- For regression: Mean Squared Error (MSE)

$$L = \frac{1}{n} \sum (y_{\text{pred}} - y_{\text{true}})^2$$

- For classification: Cross-entropy loss

$$\text{Loss} = - \sum_{i=1}^{\text{output size}} y_i \cdot \log \hat{y}_i$$

Gradient Descent

An optimization algorithm to **minimize loss** by updating weights.

$$w := w - \eta \frac{\partial L}{\partial w}$$

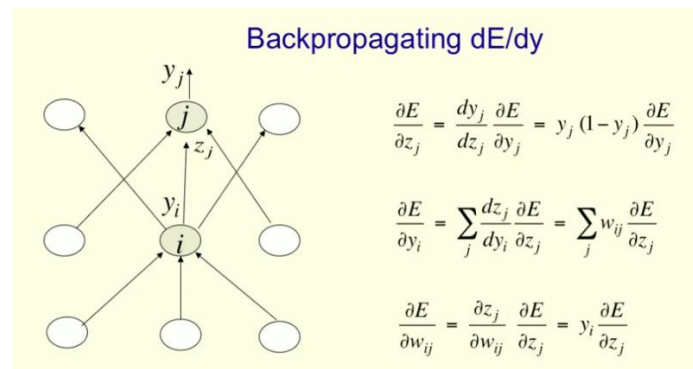
where η = learning rate.

Helps in optimizing weights by taking small steps in direction of decreasing loss.

Backpropagation

Method to compute gradients efficiently via chain rule. It Propagates the error backward from output to input. Adjust weights so the network makes better predictions.

Key concept:



using derivatives and chain rule.

Logistic Regression:

Logistic regression is a statistical classification algorithm used to predict binary outcomes e.g., yes/no, 0/1. Instead of predicting a continuous value like linear regression, it estimates the probability of an input belonging to a class using the sigmoid function.

Sigmoid Function

The logistic or sigmoid function maps real values into the range (0, 1), which makes it ideal for probability outputs:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Here z is a linear combination of input features and weights. The sigmoid function squashes the raw value z so that if z is large positive $\rightarrow$ output near 1, and if z is large negative $\rightarrow$ output near 0.

Cost (Loss) Function

To train logistic regression, we use a cost function that measures how well our model predicts the correct labels. For logistic regression, the **cross-entropy loss** is common:

$$J(w) = -\frac{1}{m} \sum_{i=1}^m (y^{(i)} \ln(\hat{y}^{(i)}) + (1 - y^{(i)}) \ln(1 - \hat{y}^{(i)}))$$

Where:

- m = number of training samples
- $y^{(i)}$ = true label
- $\hat{y}^{(i)}$ = predicted probability from sigmoid

This loss penalizes predictions that are far from the actual labels.

To minimize the cost and optimize the weights we use **Gradient Descent**.

Kalman Filters

Need of Recursive Filters

Financial markets generate **high-frequency, noisy data** such as prices, spreads, returns, and volumes.

Trading systems, especially **algorithmic and high-frequency strategies** must process this data in real time to estimate trends, fair value, or latent variables like volatility.

Recomputing statistics from scratch using all historical data is **computationally inefficient** and introduces latency. Recursive filters allow traders to **update estimates online**, using only the previous estimate and the latest observation.

Batch Average

The simple average of k data points $X_1, X_2, \dots, X_k$ is:

$$\bar{X}_k = \frac{1}{k} \sum_{i=1}^k X_i$$

This requires all past data and thus is **inefficient** as it requires more memory and computational power.

Recursive Average

You can update the average as new data arrives without revisiting all old data:

$$\bar{X}_k = \frac{k-1}{k} \bar{X}_{k-1} + \frac{1}{k} X_k$$

Define:

$$\alpha = \frac{k-1}{k} = 1 - \frac{1}{k}$$

Then the recursive average becomes:

$$\bar{X}_k = \alpha \bar{X}_{k-1} + (1 - \alpha) X_k$$

- This updates the estimate efficiently with only the old estimate and the new measurement. Hence, is more efficient.

Moving Average Filter

A *moving average* of the last n samples:

$$\bar{X}_k = \frac{1}{n} \sum_{i=k-n+1}^k X_i$$

This can also be written recursively:

$$\bar{X}_k = \bar{X}_{k-1} + \frac{X_k - X_{k-n}}{n}$$

- More responsive than a batch average, but equal weighting across all n points gives lag, especially with dynamic signals (like in case of SONAR signal readings). It helps in smoothing the noisy readings and helps in finding the curve's derivative.

First-Order Low-Pass Filter

To reduce lag and emphasize newer data, define:

$$\bar{X}_k = \alpha \bar{X}_{k-1} + (1 - \alpha) X_k$$

- Here, $0 < \alpha < 1$ is a design parameter (not tied to sample count).
- This is mathematically *identical* to the recursive average form but with constant weights, providing exponential smoothing.

- Choosing α controls the **trade-off between noise reduction** ($\alpha > 0.5$) and **responsiveness** ($\alpha < 0.5$).

Kalman Filter

The Kalman filter can be thought of as an adaptive low-pass filter where the weighting parameter changes dynamically based on uncertainty in the estimate and measurement.

Core Kalman Filter Variables

Let's define the standard Kalman notation:

- x_k : true state at time k
- z_k : measurement at time k
- $\hat{x}_{k|k-1}$: *predicted* state estimate
- $\hat{x}_{k|k}$: *updated* state estimate
- $P_{k|k-1}$: predicted error covariance
- $P_{k|k}$: updated error covariance
- A : state transition model
- H : measurement model
- Q : process (model) noise covariance
- R : measurement noise covariance
- K_k : Kalman gain

Kalman Filter System Model

State Prediction

$$\hat{x}_{k|k-1} = A \hat{x}_{k-1|k-1}$$

Covariance Prediction

$$P_{k|k-1} = A P_{k-1|k-1} A^T + Q$$

The covariance equations track how uncertain our state estimate is, and how that uncertainty changes during prediction and measurement update.

They are **covariances of the estimation error**.

Kalman Update (Measurement Update) – Fundamental Eqns

Kalman Gain

$$K_k = P_{k|k-1} H^T (H P_{k|k-1} H^T + R)^{-1}$$

Kalman gain K_k determines how much to trust the measurement vs the prediction.

High measurement noise -> trust prediction more.

High model uncertainty -> trust measurement more.

State Update

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k (z_k - H \hat{x}_{k|k-1})$$

Covariance Update

$$P_{k|k} = (I - K_k H) P_{k|k-1}$$

Uncertainty in the state estimate after incorporating the measurement

Key Concept

Kalman Filter = Optimal Recursive Estimator

It combines information from a dynamic model (how you expect the state to evolve, on the basis of historic data), and noisy measurements, using uncertainty (covariances) to weight them.

Practical Tuning

- Q and R are crucial as they reflect how noisy/uncertain the model and measurements are.
- Often tuned empirically or based on sensor specs.

Reinforcement Learning

Reinforcement Learning (RL) is a subfield of machine learning in which an agent learns to make decisions by interacting with an environment in order to **maximize cumulative reward**. Unlike supervised learning where labels are given, RL learns through **trial and feedback** from the environment.

An RL problem involves:

- **Agent:** The decision-maker e.g., robot, trading bot
- **Environment:** Everything the agent interacts with
- **State** s_t : A representation of environment at time t
- **Action** a_t : A move chosen by the agent
- **Reward** r_t : Feedback signal indicating success or failure
- **Policy** $\pi(a | s)$: A strategy mapping states to actions

The agent's **goal** is to learn a policy π that **maximizes expected total reward over time**.

Markov Decision Processes (MDPs)

An MDP provides a mathematical framework for modelling sequential decision making under uncertainty. MDPs formalize the RL problem. The key idea is that future states and rewards depend only on the **current state and action**. This means the state must encode all information from the past that is relevant for predicting future rewards and transitions.

An MDP is defined by the tuple (S, A, P, R, γ) , where:

- S is the set of states,
- A is the set of actions,
- $P(s' | s, a)$ defines the transition probabilities,
- $R(s, a)$ is the reward function,
- $\gamma \in [0,1]$ is the discount factor.

In RL, the agent learns to **optimize long-term returns**, not just immediate reward.

A policy $\pi(a | s)$ defines the agent's behavior. Given a policy, we define two central value functions. The **state-value function** measures the expected return when starting in a state and following the policy:

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s \right].$$

The **action-value function** measures the expected return when taking a specific action and then following the policy:

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a \right].$$

An optimal policy is one that maximizes expected return from every state. The corresponding optimal value functions are denoted $V^*(s)$ and $Q^*(s, a)$ and satisfy the relationship:

$$V^*(s) = \max_a Q^*(s, a).$$

This formulation makes clear that reinforcement learning is fundamentally about evaluating and comparing long-term consequences of actions under uncertainty.

Exploration & Exploitation

To learn effectively, an agent must balance:

- **Exploration:** Trying new actions to discover their effects
- **Exploitation:** Choosing known actions that maximize reward

This trade-off is key in RL because focusing only on known rewards may miss better options.

Q-Learning and the Limits of Tabular Methods

Q-learning is a model-free RL algorithm that **assigns a value to each state-action pair** and learns the optimal action values over time. The algorithm updates its estimates using rule

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right],$$

Where:

- $Q(s, a)$ – estimated quality of taking action a in state s
- α – learning rate
- γ – discount factor

Q-learning has several attractive properties:

- It does not require knowledge of the transition dynamics.
- It converges to the optimal policy under sufficient exploration.
- It is conceptually simple and easy to implement.

However, tabular Q-learning scales poorly. As the number of states or actions grows, the Q-table becomes infeasible to store or learn. This issue is commonly referred to as the **curse of dimensionality**.

In large or continuous state spaces, it becomes necessary to replace tabular representations with function approximation. This limitation motivates the development of more scalable methods, such as Deep Q-Networks

Deep Q-Networks

Tabular Q-learning becomes impractical in environments with large or continuous state spaces. In such settings, it is infeasible to store or update a separate value for every state–action pair. Deep Q-Networks (DQN) address this limitation by using a neural network to approximate the action-value function:

$$Q(s, a) \approx Q_{\theta}(s, a),$$

where θ denotes the parameters of the network.